GreenPod Labs Assignment 2

Avocado Ripening Stage Detection & Classification System

Candidate: [Enter name]

Date: [Enter submission date]

Repository: [Add GitHub URL]

Colab: [Add view-accessible Colab URL]

***Reporting rule:** Populate all result tables using actual outputs from this repository. Do not invent accuracy, F1, inference time, dataset counts, or observations before running the dataset.*

1. Introduction

Avocado ripeness sorting can reduce waste and help route fruit to appropriate handling or retail channels. This project builds an image classification pipeline for four visual stages: Underripe/Hard, Breaking/Firm, Ripe/Soft, and Overripe/Decay. Image appearance is an imperfect proxy for internal firmness, so model outputs should support—not replace—appropriate quality-control decisions.

2. Objective

Build and compare at least two approaches: (1) classical image features with SVM and Random Forest and (2) MobileNetV2 transfer learning. Analyze dataset composition, image quality and possible bias, evaluate on a held-out test set, and demonstrate inference on an example image.

3. Dataset Understanding

The assignment dataset is linked in the README. After downloading, the loader enumerates supported image formats and maps common class aliases. Record:

• Total valid images: [generated by EDA]

• Counts by class: [insert EDA table]

• Image width/height distribution: [insert EDA table/plot]

• Corrupt or unreadable files: [insert count and paths]

• Data provenance and any duplicate/session information: [document what is known]

Do not assume the dataset contains a particular number of images until it has been inspected.

4. Data Cleaning and Preprocessing

Images that cannot be decoded are logged for review. Classical features use a consistent 128×128 image representation and optional mild denoising. MobileNetV2 inputs are resized to 224×224 and normalized with ImageNet channel statistics. Augmentation is applied only to training images. Validation and test images are not augmented.

Splitting is stratified into 70% train, 15% validation, and 15% test using seed 42. For multiple views of the same fruit, or near-duplicates, split by fruit/session/group rather than randomly to prevent leakage.

5. Class Imbalance Analysis

EDA plots per-class counts. Imbalance is considered material when class counts differ substantially; report the exact ratios after running EDA. SVM and Random Forest use class-weight balancing, while evaluation includes macro-averaged metrics so each class contributes equally. If a class has too few samples for stratification or CV, gather more data or document a lower fold count/split adjustment.

6. Outlier and Bias Analysis

Review candidates for:

• Unusual brightness or darkness, strong shadows, glare/specular highlights

• Resolution and aspect-ratio variation

• Background/camera-distance differences

• Extreme color statistics

• Duplicate images, mislabeled samples, occlusion, blur, and partial fruit views

Automated brightness/color outlier rules are screening heuristics, not ground truth. A human should inspect flagged samples before excluding or relabeling. Possible confounders include cultivar, camera, background, maturity label subjectivity, and acquisition location. The report should state which biases were actually observed, rather than presenting anticipated biases as measured findings.

7. Approach 1: Classical CV + Machine Learning

RGB, HSV, and LAB channel histograms summarize color distributions. LAB a-channel statistics help characterize green-to-red color variation but are not sufficient alone to determine ripeness. Local Binary Patterns capture local texture, and GLCM properties describe contrast, homogeneity, energy, correlation, and angular second moment. The combined vector is evaluated with:

• SVM (RBF kernel; tuned C and gamma, standardized features)

• Random Forest (tuned tree count, depth and leaf size)

Hyperparameters are selected by stratified cross-validation on the training data using macro F1. Validation data supports diagnostics; test metrics are reserved for final comparison.

8. Approach 2: MobileNetV2 Transfer Learning

MobileNetV2 uses ImageNet pretrained visual representations. The feature backbone is frozen initially, and a new classifier head includes dropout, linear layers, batch normalization, and ReLU. Training uses cross-entropy, Adam, a validation-loss learning-rate scheduler, data augmentation, and early stopping. Save the best validation-loss checkpoint and evaluate it once on the test set.

9. Evaluation Metrics

• **Accuracy:** fraction of all test examples classified correctly; can hide poor minority-class performance.

• **Macro precision:** average precision across classes, measuring reliability of positive predictions for each class.

• **Macro recall:** average sensitivity across classes; important when missing a ripe/overripe stage is costly.

• **Macro F1:** harmonic mean of precision and recall, averaged equally across classes; primary selection metric when classes may be imbalanced.

• **Confusion matrix:** reveals which adjacent stages are confused.

• **ROC-AUC OVR:** optional probability-ranking metric when all classes and usable probabilities are present.

10. Model Evaluation and Comparison

Populate with `outputs/metrics/model_comparison.csv` after training.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Training Time (s)	Inference (ms/image)
SVM	[actual]	[actual]	[actual]	[actual]	[actual]	[measure]
Random Forest	[actual]	[actual]	[actual]	[actual]	[actual]	[measure]
MobileNetV2	[actual]	[actual]	[actual]	[actual]	[actual]	[measure]

The best candidate is selected by macro F1, then reviewed for class-wise recall, confusion patterns, latency, and memory footprint. A small numerical lead should not outweigh unacceptable recall for a critical stage. Include hardware, batch size, warm-up, and timing method when reporting latency.

11. Example Run Demonstration

Run `python main_demo.py --image `. Record the input filename, model predictions, confidence/probability when available, and whether the predictions agree. Include a screenshot of the displayed image and output. Do not use a training image as the only demo; use an independent test example where possible.

12. Production Considerations (Mandatory Q1)

How does the model perform in production? Performance must be measured on representative target hardware and real conveyor conditions; no production accuracy or latency can be asserted before benchmarking. MobileNetV2 is designed to be lightweight, but throughput depends on hardware, preprocessing, image resolution, batch size, and transfer overhead. Measure warmed-up single-image and batch latency on CPU/GPU/edge hardware. Scalability can be achieved with a queue-based inference service and batching, but production requires camera calibration, consistent illumination, drift monitoring, logging, and human/manual fallback. Environmental shifts such as glare,

shadows, wet skin, cultivar variation, and background changes may reduce accuracy. Real-time suitability is a measured service-level requirement, not assumed from architecture alone. Energy impact should be estimated by measuring device power and processing throughput; edge inference may reduce network transfer but is not automatically lower-carbon.

13. Drawbacks (Mandatory Q2)

• **Color ambiguity:** skin color does not always reflect internal softness; some cultivars remain green when ripe.

• **Single view:** one camera may miss localized bruises or defects on the far side.

• **Class overlap:** breaking and ripe stages form a continuous transition and labels can be subjective.

• **Dataset bias:** limited varieties, lighting, backgrounds, and cameras may not represent production conditions.

• **Label noise:** incorrect or inconsistent stage annotations cap achievable performance.

• **Feature/model limitations:** classical color-texture features can be sensitive to lighting, while a CNN can learn spurious background cues and may be overconfident.

14. Improvements with More Time (Mandatory Q3)

Collect a larger and more balanced dataset across cultivars, orchards, seasons, cameras, and lighting; use double-reviewed labels and group-based splits. Add multi-camera or rotating-fruit inspection. Explore non-destructive NIR, hyperspectral imaging, and firmness sensing for internal properties. Calibrate confidence and define abstention/manual review thresholds. Export to ONNX, benchmark runtime providers, and consider TensorRT or INT8 quantization only after checking accuracy loss. Pilot on an edge device, measure energy/latency, monitor drift, and implement model/version rollback.

15. Where Most Time Was Spent (Mandatory Q4)

Replace estimates with your actual work log. Typical major effort areas are: (1) EDA and verifying class labels/quality; (2) feature engineering and tuning color/texture descriptors; (3) checking class imbalance and reviewing questionable images; (4) augmentation and training stability for MobileNetV2; and (5) repeated model benchmarking and error analysis. State which steps took the most time in this run and why, with evidence from notebook logs.

16. Conclusion

This project provides two distinct approaches and a common, reproducible evaluation protocol. Final conclusions must be based on measured test-set metrics and qualitative error review. The most suitable model is the one that balances macro F1, class-specific recall, robustness, inference cost, and operational constraints—not simply the highest training accuracy.